



复习课（七） （机器无关优化）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



优化的来源



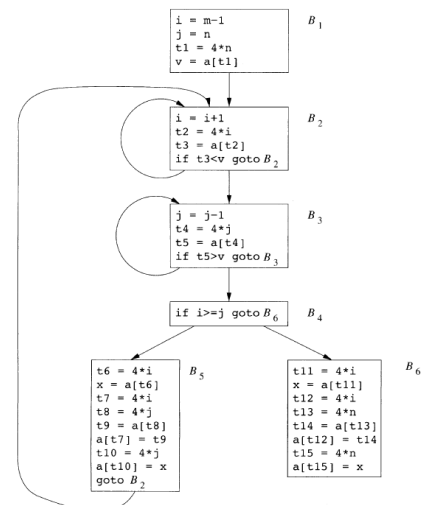
- （一）优化主要来源
- 编译器只能通过一些相对低层的语义等价转换来优化代码
- 冗余运算的原因
 - 源程序中的冗余
 - 高级程序设计语言编程的副产品
 - 比如 $A[i][j].f = 0; A[i][j].k = 1;$ 中的冗余运算
- 语义不变的优化
 - 公共子表达式消除
 - 复制传播
 - 死代码消除
 - 常量折叠



优化的来源



- （一）优化主要来源
- Quicksort 的流程图
 - B_2
 - B_3
 - B_2 、 B_3 、 B_4 、 B_5



引言



- 代码优化
 - 在目标代码中 **消除** 不必要的指令
 - 把一个指令序列 **替换** 为一个完成相同功能的更快的指令序列
- 全局优化
- 基于数据流分析技术
 - 用以收集程序相关信息的算法



优化的来源



- （一）优化主要来源
- 三地址代码

(1)	i = m-1	(16)	t7 = 4*i
(2)	j = n	(17)	t8 = 4*j
(3)	t1 = 4*n	(18)	t9 = a[t8]
(4)	v = a[t1]	(19)	a[t7] = t9
(5)	i = i+1	(20)	t10 = 4*j
(6)	t2 = 4*i	(21)	a[t10] = x
(7)	t3 = a[t2]	(22)	goto (5)
(8)	if t3 < v goto (5)	(23)	t11 = 4*i
(9)	j = j-1	(24)	x = a[t11]
(10)	t4 = 4*j	(25)	t12 = 4*i
(11)	t5 = a[t4]	(26)	t13 = 4*n
(12)	if t5 > v goto (9)	(27)	t14 = a[t13]
(13)	if i >= j goto (23)	(28)	a[t12] = t14
(14)	t6 = 4*i	(29)	t15 = 4*n
(15)	x = a[t6]	(30)	a[t15] = x



数据流分析



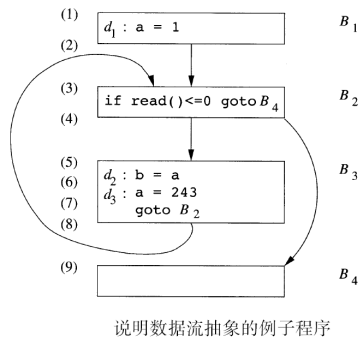
- 数据流分析
 - 用于获取数据沿着程序执行路径流动的信息的相关技术
 - 是优化的基础
- 例如
 - 两个表达式是否一定计算得到相同的值？（公共子表达式）
 - 一个语句的计算结果有没有可能被后续语句使用？（死代码消除）



- 数据流抽象
- 程序点
 - 三地址语句之前或之后的位置
 - 基本块内部：一个语句之后的程序点等于下一个语句之前的程序点
 - 如果流图中有 B_1 到 B_2 的边，那么 B_2 的第一个语句之前的点可能紧跟在 B_1 的最后语句之后的点后面执行
- 从 p_1 到 p_2 的执行路径： $p_1, p_2, \dots, p_n$
 - 要么 p_i 是一个语句之前的点，且 p_{i+1} 是该语句之后的点
 - 要么 p_i 是某个基本块的结尾，且 p_{i+1} 是该基本块的某个后继的开头



- 示例：
- 路径
 - 1, 2, 3, 4, 9
 - 1, 2, 3, 4, 5, 6, 7, 8, 9
 - ...
- 第一次到达(5)， $a=1$ ；第二次到达(5)， $a=243$ ；且之后都是243
- 我们可以说：
 - 点(5)具有特性 $a=1$ or $a=243$
 - 表示成为 $\langle a, \{1, 243\} \rangle$



- 基本块的控制流非常简单
 - 从头到尾不会中断
 - 没有分支
- 基本块的效果就是各个语句的效果的复合
- 可以预先处理基本块内部的数据流关系，给出基本块对应的传递函数；

$$IN[B] = f_B(OUT[B]) \quad \text{或者} \quad OUT[B] = f_B(IN[B])$$
- 设基本块包含语句 $s_1, s_2, \dots, s_n$

$$f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$$



- 数据流抽象
- 出现在某个程序点的程序状态
 - 在某个运行时刻，当指令指针指向这个程序点时，各个变量和动态内存中存放的值
 - 指令指针可能多次指向同一个程序点
 - 因此一个程序点可能对应多个程序状态
- 数据流分析把可能出现在某个程序点上的程序状态集合总结为一些特性
 - 不管程序怎么运行，当它到达某个程序点时，程序状态总是满足分析得到的特性
 - 不同的分析技术关心不同的信息
- 为了高效、自动地进行数据流分析，通常要求这些特性能够被高效地表示和求解



- 数据流分析模式
- 数据流分析：对一组约束求解
 - $IN[s]$ 和 $OUT[s]$
- 基于语句语义的约束(传递函数)
 - $IN[s] = f_s(OUT[s])$ (逆向)
 - $OUT[s] = f_s(IN[s])$ (正向)
- 基于控制流的约束
 - $IN[s_{i+1}] = OUT[s_i]$



- 到达定值分析
- 活跃变量分析
- 可用表达式分析



数据流分析：到达定值

- 到达定值
 - 如果存在一条从定值d后面的程序点到达某个点p的路径，且这条路径上d没有被杀死，那么定值d到达p
- 杀死：路径上对x的其他定值杀死了之前对x的定值
- 直观含义
 - 如果d到达p，那么在p点使用的值可能就是由d定值的
- 思考：不确定是否赋值该怎么办？
 - $*p = 3$ （不确定p的指向）
 - 过程参数、数组、指针等等



数据流分析：到达定值

- 语句/基本块的传递方程（1）
- 定值 $d: u=v+w$
 - 生成了对变量u的定值d，杀死了其他对u的定值
 - 生成-杀死：
 - $OUT[B] = f_B(IN[B])$
 - $f_d(x) = gen_d \cup (x - kill_d)$
 - 把 $x = IN[B]$ 带入
 - 特别地，其中 $gen_d = \{d\}$, $kill_d = \{\text{程序中其他对u的定值}\}$



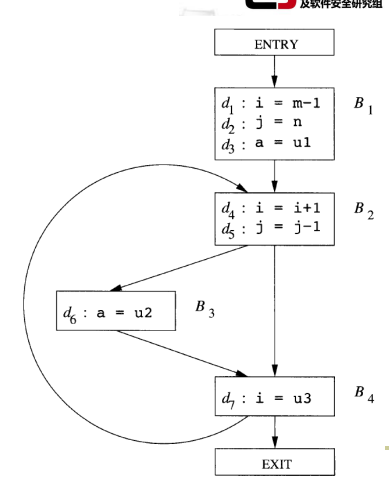
数据流分析：到达定值

- 语句/基本块的传递方程（2）
- 设B有n个语句，第i个语句的传递函数为 f_i
- $f_B(x) = gen_B \cup (x - kill_B)$
- $gen_B = gen_n \cup (gen_{n-1} - kill_n) \cup (gen_{n-2} - kill_{n-1} - kill_n) \cup (gen_1 - kill_2 - kill_3 \dots - kill_n)$
- $kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n$
- $kill_B$ 为被B各个语句杀死的定值的并集
- gen_B 是被第i个语句生成，且没有被其后的句子杀死的定值的集合



数据流分析：到达定值

- 示例：
 - B_1 中全部定值到达 B_2 的开头
 - d_5 到达 B_2 的开头（循环）
 - d_2 被 d_5 杀死，不能到达 B_3 、 B_4 的开头
 - d_4 不能到达 B_2 的开头，因为被 d_7 杀死
 - d_6 到达 B_2 的开头



数据流分析：到达定值

- 语句/基本块的传递方程（1）
- $OUT[B] = gen_d \cup (IN[B] - kill_d)$
- 生成-杀死形式的函数的并置（复合）
 - $f_2(f_1(x)) = gen_2 \cup (gen_1 \cup (x - kill_1) - kill_2)$
 $= (gen_2 \cup (gen_1 - kill_2)) \cup (x - kill_1 \cup kill_2)$
 - 生成的定值：由第二部分生成、以及由第一个部分生成且没有被第二部分杀死
 - 杀死的定值：被第一部分杀死的定值、以及被第二部分杀死的定值



数据流分析：到达定值

- 到达定值的控制流方程：
- 只要一个定值能够沿某条路径到达一个程序点，这个定值就是到达定值
- $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱基本块}} OUT[P]$
 - 如果从基本块P到B有一条控制流边，那么 $OUT[P]$ 在 $IN[B]$ 中
 - 一个定值必然先在某个前驱的OUT值中，才能出现在B的IN中
- $\cup$ 称为到达定值的交汇运算符



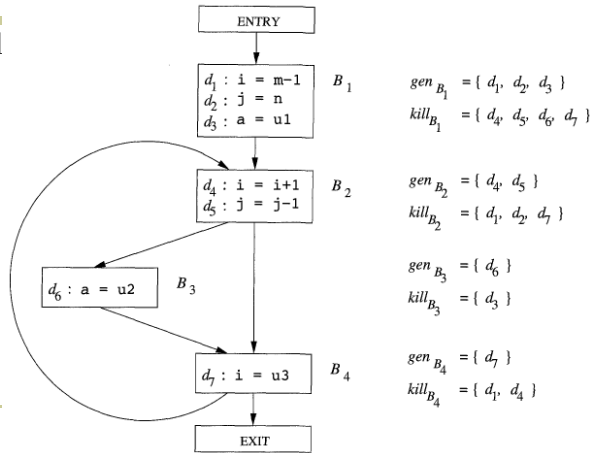
数据流分析：到达定值

- 控制流方程的迭代解法（1）：
- ENTRY基本块的传递函数是常函数
 $OUT[ENTRY] = \text{空集}$
- 其他基本块
 $OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$
 $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱基本块}} OUT[P]$
- 迭代解法
 - 首先求出各基本块的gen和kill
 - 令所有的OUT[B]都是空集，然后不停迭代，得到最小不动点的解

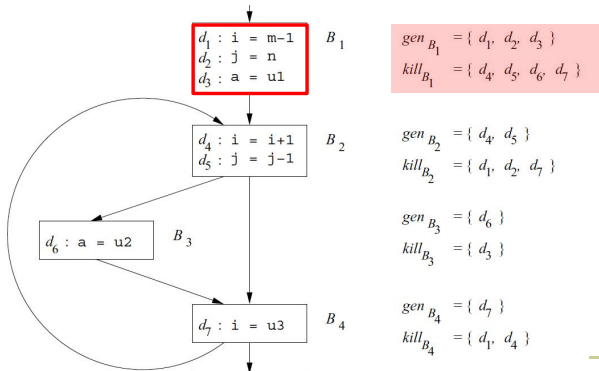


数据流分析：到达定值

■ gen和kil

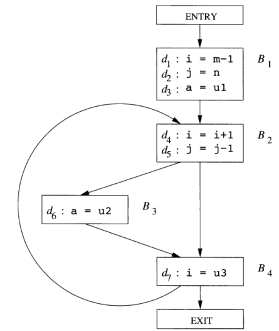


数据流分析：到达定值

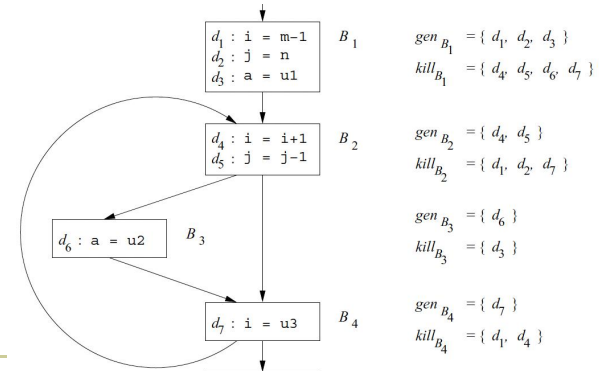


数据流分析：到达定值

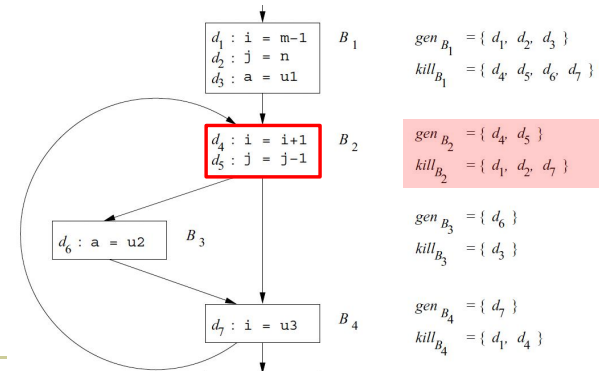
- 控制流方程的迭代解法（2）：
- 输入：流图、各基本块的kill和gen集合
- 输出：IN[B] 和 OUT[B]
- 方法：
 - 1) $OUT[ENTRY] = \emptyset$;
 - 2) **for** (除 ENTRY 之外的每个基本块 B) $OUT[B] = \emptyset$;
 - 3) **while** (某个 OUT 值发生了改变)
 - 4) **for** (除 ENTRY 之外的每个基本块 B) {
 - 5) $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$;
 - 6) $OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$;
 - }



数据流分析：到达定值

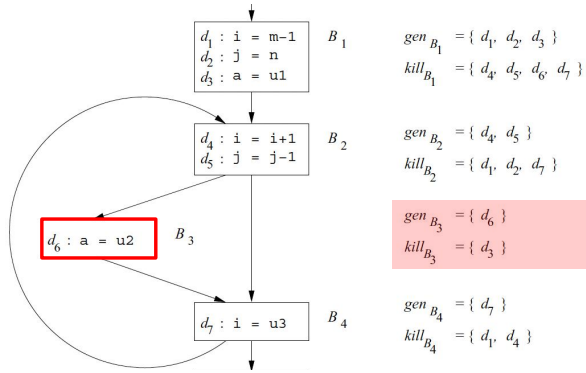


数据流分析：到达定值

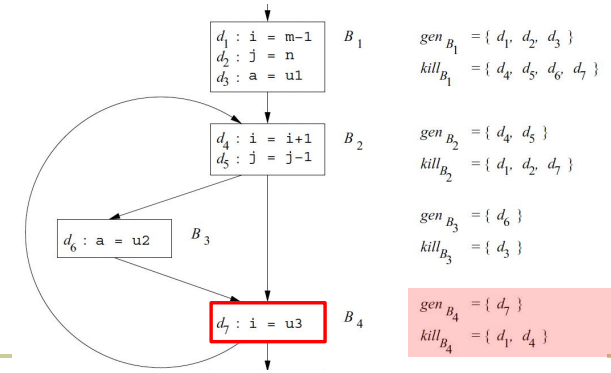




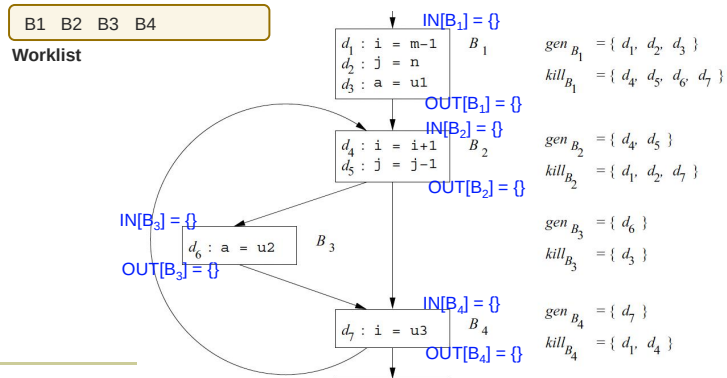
数据流分析：到达定值



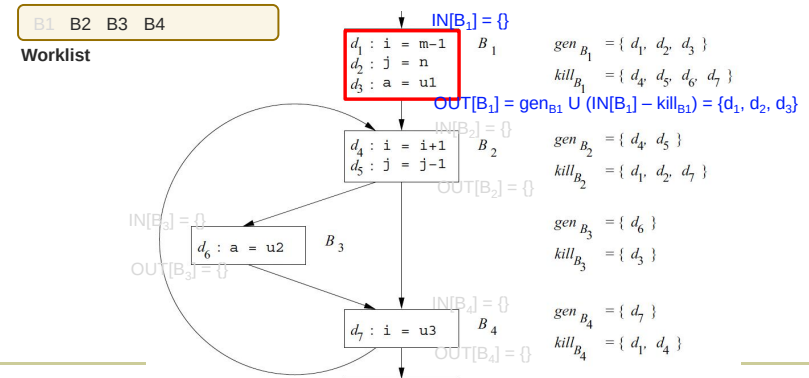
数据流分析：到达定值



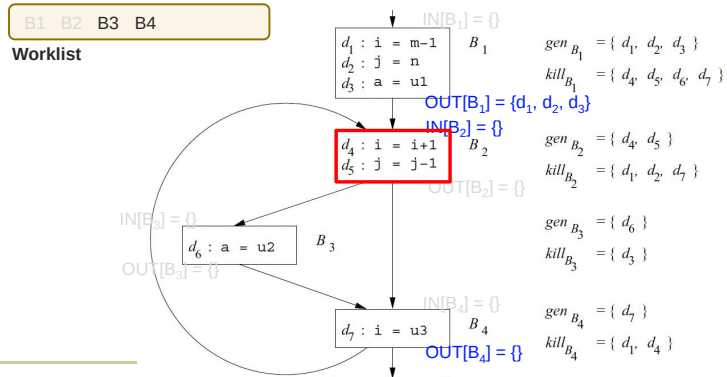
数据流分析：到达定值



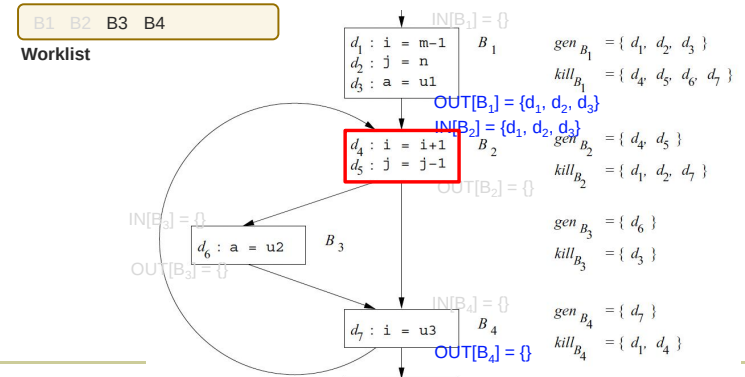
数据流分析：到达定值



数据流分析：到达定值



数据流分析：到达定值

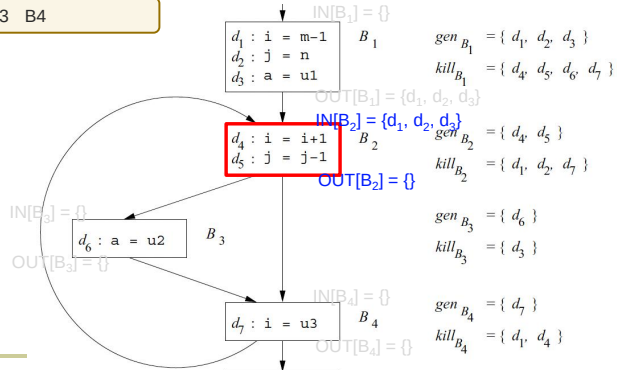




数据流分析：到达定值

B1 B2 B3 B4

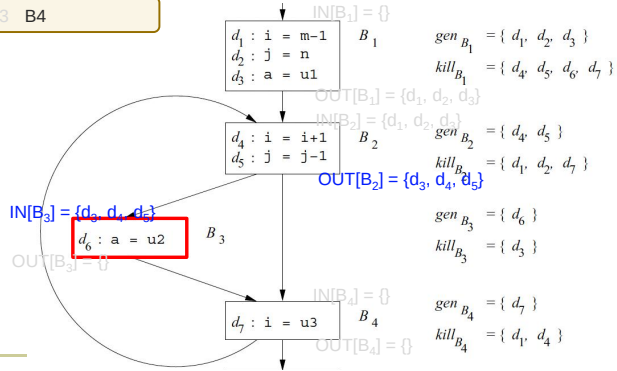
Worklist



数据流分析：到达定值

B1 B2 B3 B4

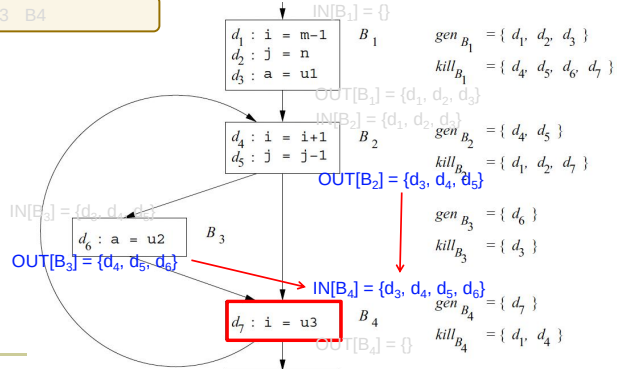
Worklist



数据流分析：到达定值

B1 B2 B3 B4

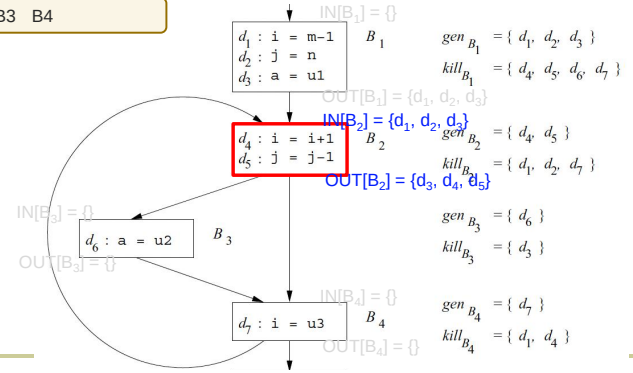
Worklist



数据流分析：到达定值

B1 B2 B3 B4

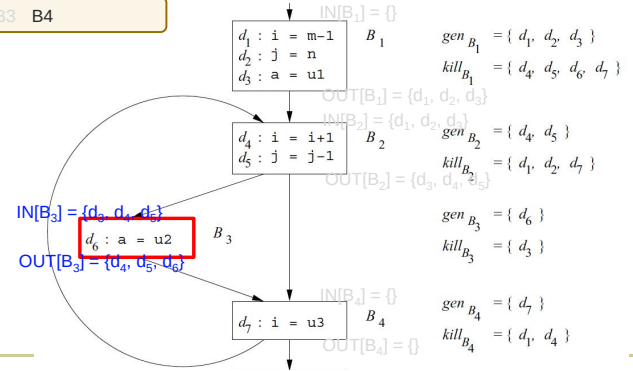
Worklist



数据流分析：到达定值

B1 B2 B3 B4

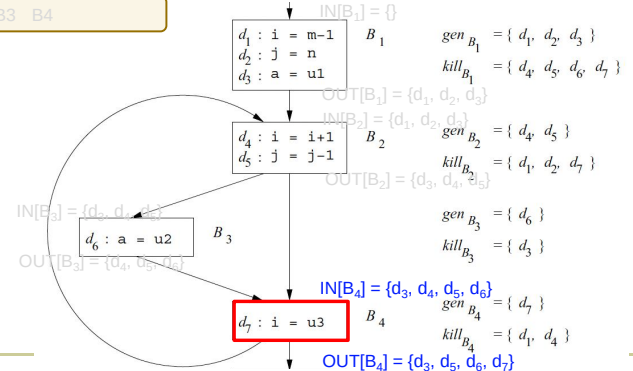
Worklist



数据流分析：到达定值

B1 B2 B3 B4

Worklist

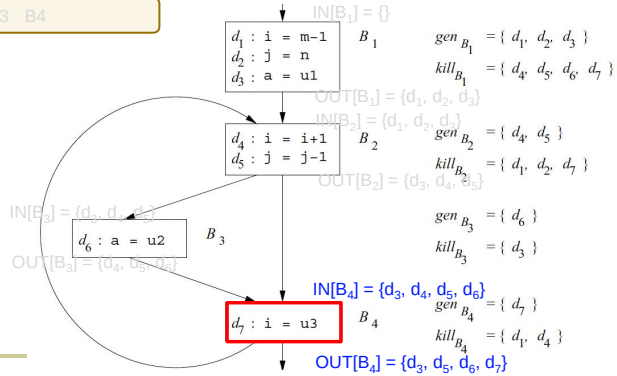




数据流分析：到达定值

B1 B2 B3 B4

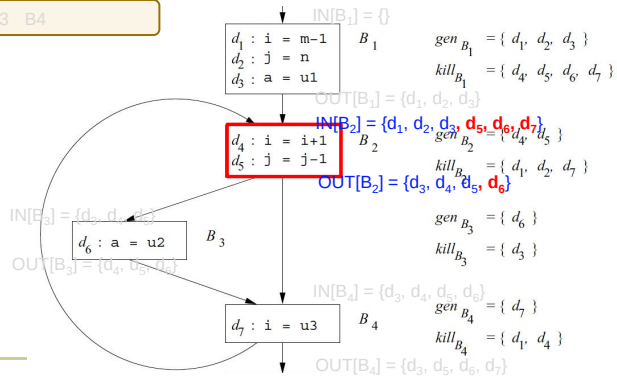
Worklist



数据流分析：到达定值

B1 B2 B3 B4

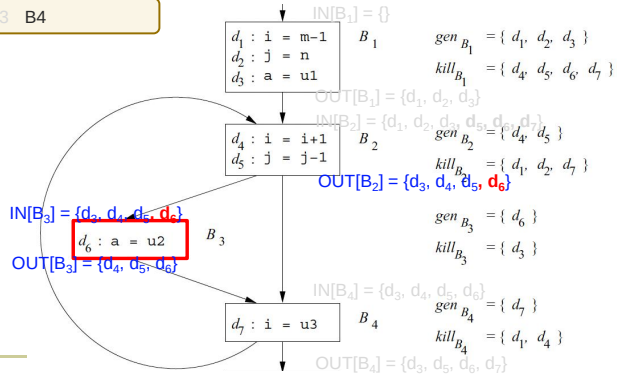
Worklist



数据流分析：到达定值

B1 B2 B3 B4

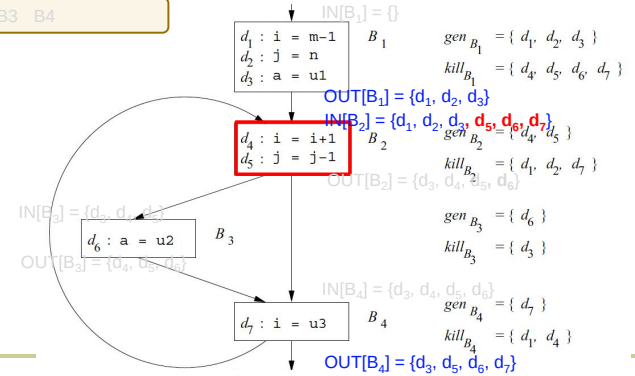
Worklist



数据流分析：到达定值

B1 B2 B3 B4

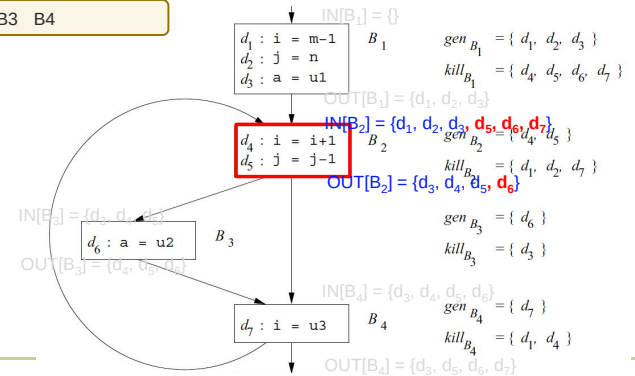
Worklist



数据流分析：到达定值

B1 B2 B3 B4

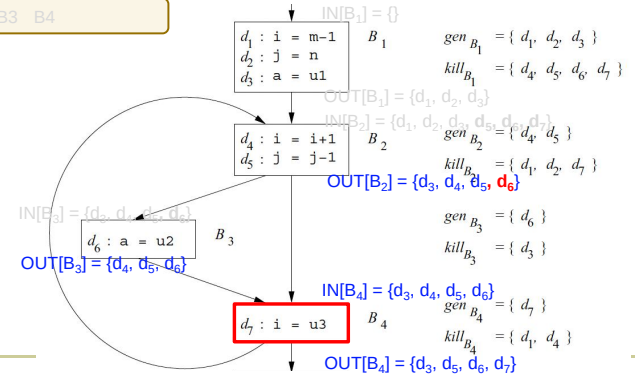
Worklist



数据流分析：到达定值

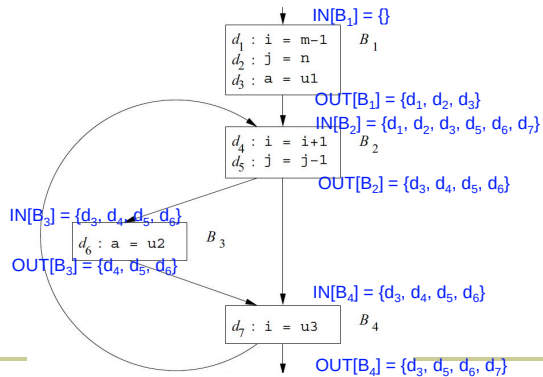
B1 B2 B3 B4

Worklist





数据流分析：到达定值



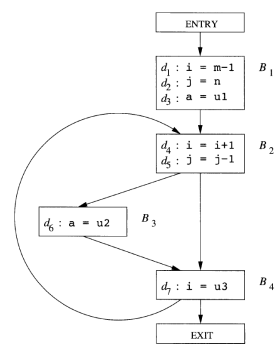
数据流分析：到达定值

- 控制流方程的迭代解法（3）
- 解法的正确性
 - 直观解释：不断向前传递各个定值，直到该定值被杀死为止
- 算法为什么能终止？
 - 各个OUT[B]在算法执行过程中不会变小
 - 且OUT[B]显然有有穷的上界
 - 只有一次迭代之后增大了某个OUT[B]的值，算法才会进行下一次迭代
- 最大迭代次数是流图的结点数n
 - 定值经过n步必然已经到达所有可能到达的结点
- 算法结束时，各个OUT/IN值必然满足数据流方程



数据流分析：到达定值

- 到达定值求解示例
- 7个bit从左到右表示 $d_1, d_2, \dots, d_n$
- for循环时依次遍历 B_1, B_2, B_3, B_4
- 每一列表示一次迭代计算；
- B_1 生成 d_1, d_2, d_3 ，杀死 d_4, d_5, d_6, d_7
- B_2 生成 d_4, d_5 ，杀死 d_1, d_2, d_7
- B_3 生成 d_6 ，杀死 d_3
- B_4 生成 d_7 ，杀死 d_1, d_4



Block B	OUT[B] ⁰	IN[B] ¹	OUT[B] ¹	IN[B] ²	OUT[B] ²
B ₁	000 0000	000 0000	111 0000	000 0000	111 0000
B ₂	000 0000	111 0000	001 1100	111 0111	001 1110
B ₃	000 0000	001 1100	000 1110	001 1110	000 1110
B ₄	000 0000	001 1110	001 0111	001 1110	001 0111
EXIT	000 0000	001 0111	001 0111	001 0111	001 0111

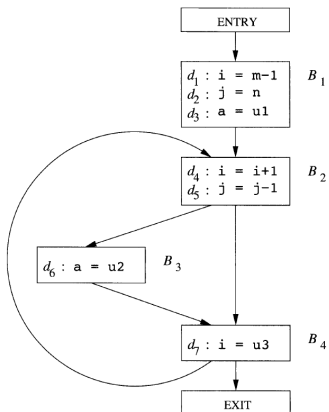


数据流分析：活跃变量分析

- 活跃变量分析
 - x在p上的值是否会在某条从p出发的路径中使用
 - 一个变量x在p上活跃，当且仅当存在一条从p点开始的路径，该路径的末端使用了x，且路径上没有对x进行覆盖
- 用途
 - 寄存器分配/死代码删除/无用赋值...
 - （考虑我们前面讲的优化）
- 数据流值
 - （活跃）变量的集合



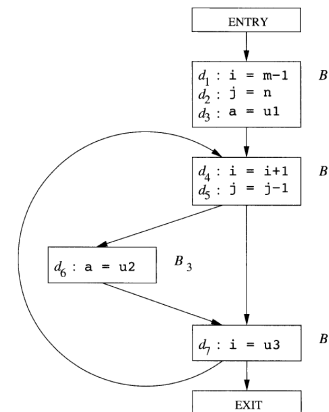
数据流分析：活跃变量分析



OUT[B]	B1	B2	B3	B4
a				
i				
j				
m				
n				
u1				
u2				
u3				



数据流分析：活跃变量分析



OUT[B]	B1	B2	B3	B4
a	x	x	x	x
i	√	x	x	√
j	√	√	√	√
m	x	x	x	x
n	x	x	x	x
u1	x	x	x	x
u2	√	√	√	√
u3	√	√	√	√



- 基本块内的数据流方程:
- 基本块的传递函数仍然是生成-杀死形式, 但是从OUT值计算出IN值 (逆向)
- $IN[B] = f_B(OUT[B])$
- $f_B(x) = use_B \cup (x - def_B)$
 - use_B , 在基本块B中引用, 但是引用前在B中没有被定值的集合; 可能在B中先于定值被使用 (GEN)
 - def_B , 在基本块中定值, 但是定值前在B中没有被引用的变量的集合; 在B中一定先于使用被定值 (KILL)



- USEB和DEFB的用法:
- 语句的传递函数
 - $s: x = y + z$
 - $use_s = \{y, z\}$
 - $def_s = \{x\} - \{y, z\}$ // $x = y + z$ 是模板, y, z 可能和 x 相同
- 假设基本块中包含语句 $s_1, s_2, \dots, s_n$, 那么
- $use_B = use_{s_1} \cup (use_{s_2} - def_{s_1}) \cup (use_{s_3} - def_{s_1} - def_{s_2}) \cup \dots \cup (use_{s_n} - def_{s_1} - def_{s_2} - \dots - def_{s_{n-1}})$
- $def_B = def_{s_1} \cup def_{s_2} \cup \dots \cup def_{s_n}$



输入: 流图G, 各个基本块B的use和def
输出: $IN[B]$ 和 $OUT[B]$

```

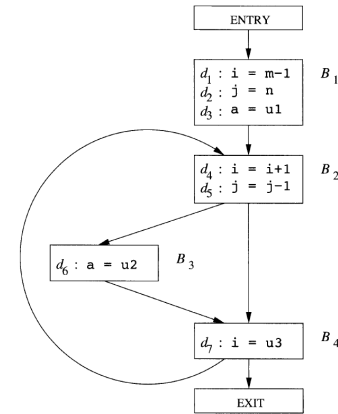
IN[EXIT] = ∅;
for (除 EXIT 之外的每个基本块 B) IN[B] = ∅;
while (某个 IN 值发生了改变)
  for (除 EXIT 之外的每个基本块 B) {
    OUT[B] = ∪S 是 B 的一个后继 IN[S];
    IN[B] = use_B ∪ (OUT[B] - def_B);
  }

```



- 活跃变量分析的迭代方法

$\triangleright use_{B_1} = \{m, n, u1\}$
 $\triangleright def_{B_1} = \{i, j, a\}$
 $\triangleright use_{B_2} = \{i, j\}$
 $\triangleright def_{B_2} = \Phi$
 $\triangleright use_{B_3} = \{u2\}$
 $\triangleright def_{B_3} = \{a\}$
 $\triangleright use_{B_4} = \{u3\}$
 $\triangleright def_{B_4} = \{i\}$



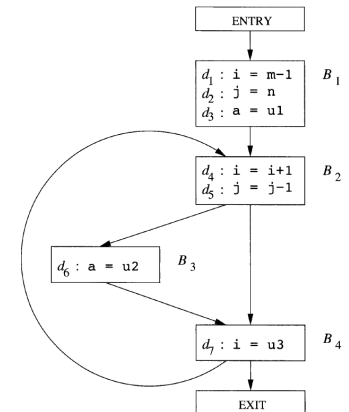
- 活跃变量数据流方程:
- 任何变量在程序出口处不再活跃
 - $IN[EXIT] = \text{空集}$
- 对于所有不等于EXIT的基本块
 - $IN[B] = use_B \cup (OUT[B] - def_B)$
 - $OUT[B] = \cup_{S \text{ 是 } B \text{ 的后继基本块}} IN[S]$
- 和到达定值相比较
 - 都使用并集运算 $\cup$ 作为交汇运算
 - 数据流值的传递方向相反: 因此初始化的值不一样



- 活跃变量数据流方程求解例子

	$OUT[B]^1$	$IN[B]^1$	$OUT[B]^2$	$IN[B]^2$	$OUT[B]^3$	$IN[B]^3$
B_4		$u3$	$ij, u2, u3$	$j, u2, u3$	$ij, u2, u3$	$j, u2, u3$
B_3	$u3$	$u2, u3$	$j, u2, u3$	$j, u2, u3$	$j, u2, u3$	$j, u2, u3$
B_2	$u2, u3$	$ij, u2, u3$	$j, u2, u3$	$ij, u2, u3$	$j, u2, u3$	$ij, u2, u3$
B_1	$ij, u2, u3$	$m, n, u1, u2, u3$	$ij, u2, u3$	$m, n, u1, u2, u3$	$ij, u2, u3$	$m, n, u1, u2, u3$

$\triangleright use_{B_1} = \{m, n, u1\}$
 $\triangleright def_{B_1} = \{i, j, a\}$
 $\triangleright use_{B_2} = \{i, j\}$
 $\triangleright def_{B_2} = \Phi$
 $\triangleright use_{B_3} = \{u2\}$
 $\triangleright def_{B_3} = \{a\}$
 $\triangleright use_{B_4} = \{u3\}$
 $\triangleright def_{B_4} = \{i\}$



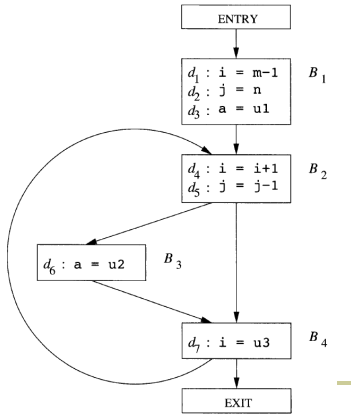


活跃变量数据流方程求解例子

	OUT[B] ¹	IN[B] ¹	OUT[B] ²	IN[B] ²	OUT[B] ³	IN[B] ³
B ₄		u3	i,j,u2,u3	j,u2,u3	i,j,u2,u3	j,u2,u3
B ₃	u3	u2,u3	j,u2,u3	j,u2,u3	j,u2,u3	j,u2,u3
B ₂	u2,u3	i,j,u2,u3	j,u2,u3	i,j,u2,u3	j,u2,u3	i,j,u2,u3
B ₁	i,j,u2,u3	m,n,u1,u2,u3	i,j,u2,u3	m,n,u1,u2,u3	i,j,u2,u3	m,n,u1,u2,u3

OUT[B]	B1	B2	B3	B4
a	x	x	x	x
i	√	x	x	√
j	√	√	√	√
m	x	x	x	x
n	x	x	x	x
u1	x	x	x	x
u2	√	√	√	√
u3	√	√	√	√

$\triangleright use_{B_1} = \{m, n, u1\}$
 $\triangleright def_{B_1} = \{i, j, a\}$
 $\triangleright use_{B_2} = \{i, j\}$
 $\triangleright def_{B_2} = \Phi$
 $\triangleright use_{B_3} = \{u2\}$
 $\triangleright def_{B_3} = \{a\}$
 $\triangleright use_{B_4} = \{u3\}$
 $\triangleright def_{B_4} = \{i\}$



在x的应用点u可以用y替代x的条件:

- 从流图的首节点到达u的每条路径都存在复制语句x=y
- 并且从最后一条复制语句x=y到u之间没有再次对x或者y进行定值

生成-杀死

- 杀死: 基本块对x或y赋值, 且没有重新计算x+y, 那么它杀死了x+y
- 生成: 基本块求值x+y, 且之后没有对x或者y赋值, 那么它生成了x+y

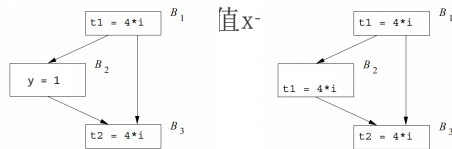


在x的应用点u可以用y替代x的条件:

- 从流图的首节点到达u的每条路径都存在复制语句x=y
- 并且从最后一条复制语句x=y到u之间没有再次对x或者y进行定值

生成-杀死

- 杀死: 基本块对x或y赋值, 且没有重新计算x+y, 那么它杀死了x+y
- 生成: 基本块求值x+y, 且之后没有对x或者y赋值, 那么它生成了x+y



4 * i is available before B₃

4 * i is available before B₃



x+y在p点可用的条件

- 从流图入口结点到达p的**每条路径**都对x+y求值, 且在最后一次求值之后再没有对x或者y赋值

主要用途

- 消除全局公共子表达式
- 复制传播

直观意义: 在点p上, x+y已经被计算过了, 不用再重新计算了

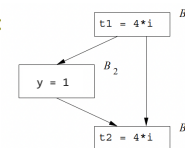


在x的应用点u可以用y替代x的条件:

- 从流图的首节点到达u的每条路径都存在复制语句x=y
- 并且从最后一条复制语句x=y到u之间没有再次对x或者y进行定值

生成-杀死

- 杀死: 基本块对x或y赋值, 且没有重新计算x+y, 那么它杀死了x+y
- 生成: 基本块求值x+y, 且之后没有对x或者y赋值, 那么它生成了x+y



4 * i is available before B₃

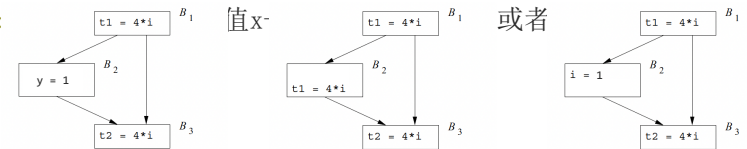


在x的应用点u可以用y替代x的条件:

- 从流图的首节点到达u的每条路径都存在复制语句x=y
- 并且从最后一条复制语句x=y到u之间没有再次对x或者y进行定值

生成-杀死

- 杀死: 基本块对x或y赋值, 且没有重新计算x+y, 那么它杀死了x+y
- 生成: 基本块求值x+y, 且之后没有对x或者y赋值, 那么它生成了x+y



4 * i is available before B₃

4 * i is available before B₃

4 * i is NOT available before B₃



- 可用表达式传递函数
 - 对于可用表达式数据流模式而言，
 - 如果基本块B对x或者y进行了（或可能进行）定值，且以后没有重新计算x+y，则称B杀死表达式x+y。
 - 如果基本块B对x+y进行计算，并且之后没有重新定值x或y，则称B生成表达式x+y
- $f_B(x) = e_gen_B \cup (x - e_kill_B)$
 - e_gen_B ：基本块B所生成的可用表达式的集合
 - e_kill_B ：基本块B所杀死的U中的表达式的集合；U表示所有出现在程序中一个或多个语句的右部的表达式的全集



- 基本块生成/杀死的表达式的例子

语 句	可用表达式
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$



- 可用表达式分析的迭代方法
- 注意：OUT值的初始化值是全集
 - 这样的初始集合可以求得更有用的解

```

OUT[ENTRY] =  $\emptyset$ ;
for (除 ENTRY 之外的每个基本块 B) OUT[B] = U;
while (某个 OUT 值发生了改变)
  for (除 ENTRY 之外的每个基本块 B) {
    IN[B] =  $\bigcap_{P \text{ 是 } B \text{ 的一个前驱}} \text{OUT}[P]$ ;
    OUT[B] =  $e\_gen_B \cup (IN[B] - e\_kill_B)$ ;
  }

```



- 计算基本块生成的表达式:
- 初始化 $S = \{ \}$
- 从头到尾逐个处理基本块中的指令 $x = y + z$
 - 把 $y + z$ 添加到 S 中；
 - 从 S 中删除任何涉及变量 x 的表达式
- 遍历结束时得到基本块生成的表达式集合；
- 杀死的表达式集合
 - 表达式的某个分量在基本块中定值，且没有被再次生成



- 可用表达式的数据流方程

- ENTRY 结点的出口处没有可用表

- $OUT[ENTRY] = \{ \}$

- 其他基本块的方程

- $OUT[B] = e_gen_B \cup (IN[B] - e_kill_B)$
- $IN[B] = \bigcap_{P \text{ 是 } B \text{ 的前驱基本块}} OUT[P]$

- 和其他方程类比

- 前向传播
- 交汇运算是交集运算

语 句	可用表达式
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$



	到达定值	活跃变量	可用表达式
域	Sets of definitions	Sets of variables	Sets of expressions
方向	Forwards	Backwards	Forwards
传递函数	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
边界条件	$OUT[ENTRY] = \emptyset$	$IN[EXIT] = \emptyset$	$OUT[ENTRY] = \emptyset$
交汇运算($\wedge$)	$\cup$	$\cup$	$\cap$
方程组	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{P, pred(B)} OUT[P]$	$IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge_{S, succ(B)} IN[S]$	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{P, pred(B)} OUT[P]$
初始值	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$



- 循环的重要性
 - 程序的大部分执行时间都花在循环上
- 相关概念
 - 支配结点
 - 深度优先排序
 - 回边
 - 图的深度
 - 可归约性
- 两个重要问题
 - 如何寻找循环
 - 数据流分析的收敛速度



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```

69



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```

70



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```

71



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```

72



流程图中的循环



1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

73



流程图中的循环



1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

75



流程图中的循环



1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

77



流程图中的循环



1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

74



流程图中的循环



1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

76



流程图中的循环



1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

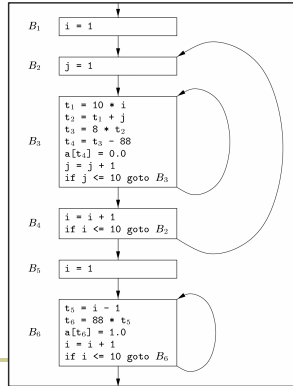
78



流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



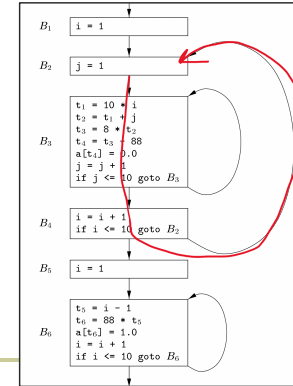
79



流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



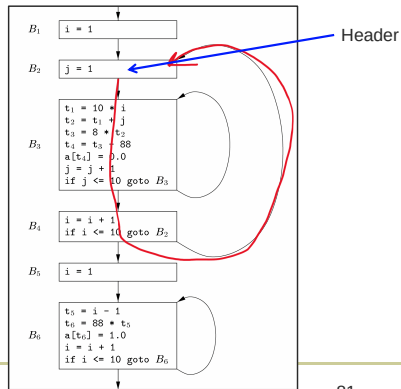
80



流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



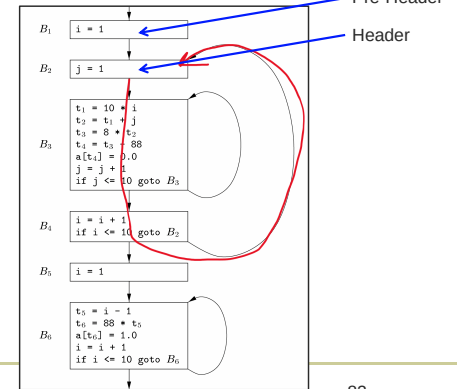
81



流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



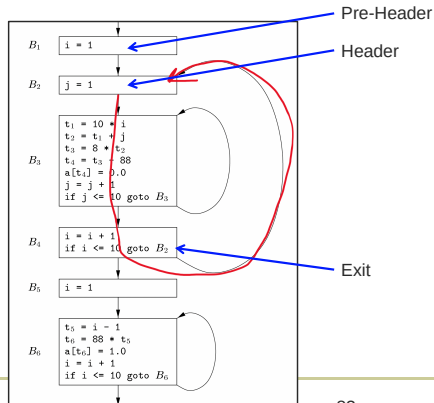
82



流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



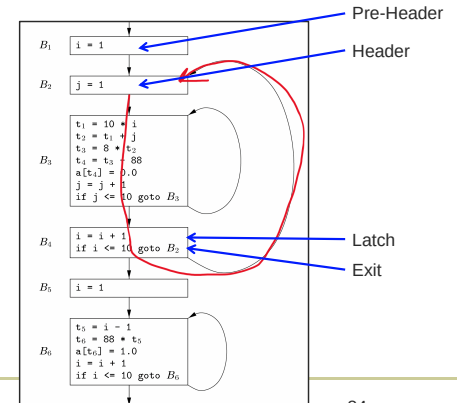
83



流程图中的循环

```

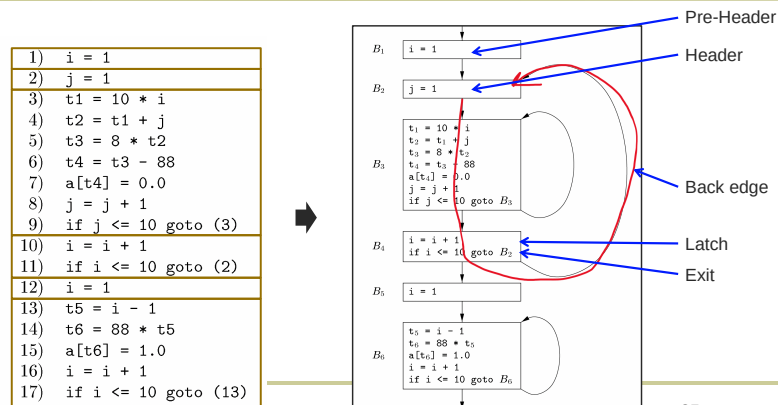
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
    
```



84



流程图中的循环



85



流程图中的循环

■ 自然循环:

- 1. 单个循环头 (header)
- 2. 最大强连通分量
 - 强连通分量: 一个有向图的强连通分量是一个最大的子图, 在这个子图中, 任意两个节点都是互相可达的。也就是说, 从节点A可以走到节点B, 从节点B也可以走回节点A
 - 在控制流图的上下文中: 一个循环正是一个强连通分量。因为你在循环里转圈, 可以从任何一点走到任何其他一点 (通过继续执行循环)
 - 最大的: 这个前缀意味着我们取的是包含该循环头在内的、最大的那个强连通分量。这确保了我们将整个循环体都包含进来, 而不仅仅是其中的一部分

87



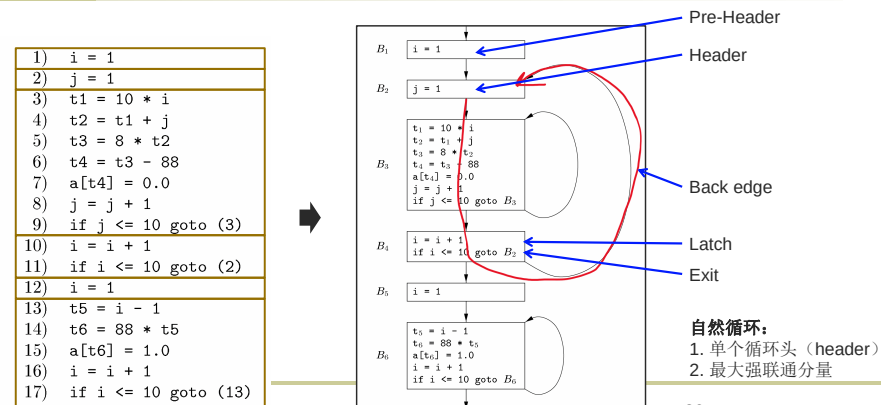
流程图中的循环

■ 支配的定义:

- A **dom** B
 - if all paths from Entry to B goes through A
- A **post-dom** B
 - if all paths from B to Exit goes through A
- **Strict** (post-)dominance – A (post-)dom B but A ≠ B
- **Immediate** dominance – A strict-dom B, but there's no C, such that A strict-dom C, C strict-dom B



流程图中的循环



86



流程图中的循环

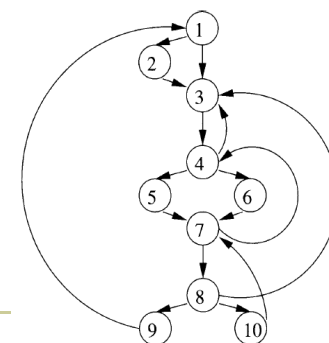
■ 自然循环:

- 自然循环的性质
 - 有一个唯一的入口结点 (循环头 header)。这个结点支配循环中的所有结点
 - 必然存在进入循环头的回边 (否则怎么构成循环?)
- 假设流图中存在两个结点d和n, 满足d dom n:
 - 自然循环的定义
 - 给定回边n→d的自然循环是d加上不经过d就能够到达n的结点的集合
 - d是这个循环的头



流程图中的循环

- 支配结点 (必经节点):
- 如果每一条从入口结点到达n的路径都经过d, 我们就说d支配 (dominate) n, 记为d dom n
- 每个节点都支配自己
- 右图:
 - 2只支配自己
 - 3支配除了1, 2之外的其它所有结点
 - 4支配1、2、3之外的其它结点
 - 5、6只支配自身
 - 7支配7, 8, 9, 10
 - 8支配8, 9, 10
 - 9, 10只支配自身

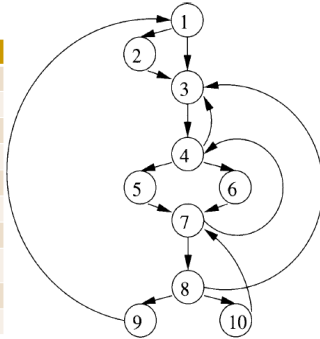




流图中的循环

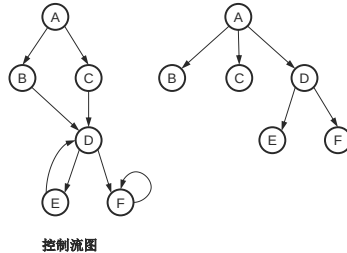
- 支配结点(必经节点):
- 如果每一条从入口结点到达n的路径都经过d, 我们就说d支配(dominate)n, 记为 $d \text{ dom } n$
- 每个节点都支配自己

支配节点	支配对象
1	1~10
2	2
3	3~10
4	4~10
5	5
6	6
7	7~10
8	8~10
9	9
10	10



流图中的循环

- 支配结点树:
 - 节点: 基本块
 - 边: 直接支配关系

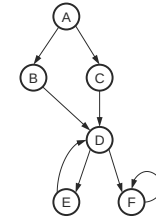


控制流图



流图中的循环

- 支配结点树:
 - 节点: 基本块
 - 边: 直接支配关系



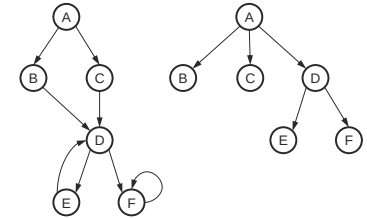
控制流图

Try!



流图中的循环

- 支配结点树:
 - 节点: 基本块
 - 边: 直接支配关系
- (最近) 公共支配者
 - 最低公共祖先
 - 程序的支配者树 就是一棵树, 在这棵树上, 两个节点的 LCA 正好就是它们在控制流图中的最近公共支配者
 - 近似线性的预处理时间
- 应用: 确定变量定义的位置



控制流图



流图中的循环

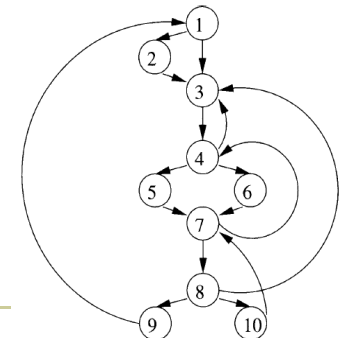
- (最近) 公共支配者
 - 最低公共祖先
 - 程序的支配者树 就是一棵树, 在这棵树上, 两个节点的 LCA 正好就是它们在控制流图中的最近公共支配者
 - 近似线性的预处理时间
- 应用: 确定变量定义的位置
 - 部分冗余消除, 目标: 消除在部分路径上重复计算的表达式
 - 静态单赋值形式的 Φ 函数插入, 目标: 在构建 SSA 形式时, 正确插入 Φ 函数

利用控制流图、支配者树和图论算法(如 LCA 查询), 智能地、自动化地为变量或表达式的计算结果选择一个最优的定义位置, 以达到消除冗余、优化代码的目的。



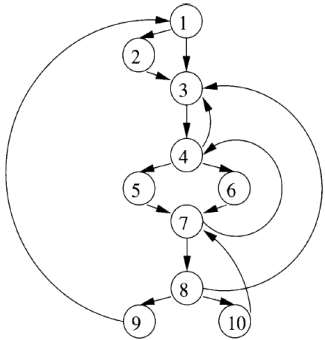
流图中的循环

- 支配结点树:
- 支配结点树可以表示支配关系
 - 根结点: 入口结点
 - 每个结点d支配且只支配树中的后代结点
- 直接支配结点
 - 从入口结点到达n的任何路径(不含n)中, 它是路径中最后一个支配n的结点
 - 前面的例子: 1直接支配3, 3直接支配4
 - n的直接支配结点m具有如下性质: 如果 $d \text{ dom } n$, 那么 $d \text{ dom } m$

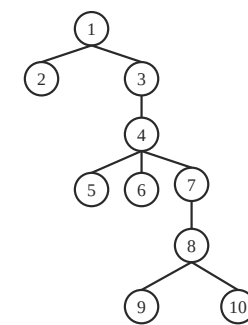
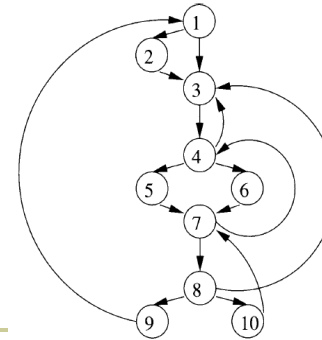




■ 支配结点树:



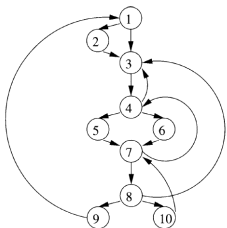
■ 支配结点树:



■ 寻找支配结点:

■ 输入: 流图G

■ 输出: 对于N中的各个节点n, 给出D(n), 即支配n的所有结点的集合.

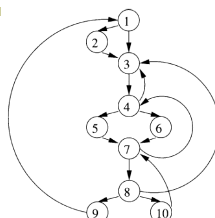


	支配结点
域	The power set of N
方向	Forwards
传递函数	$f_B(x) = x \cup \{B\}$
边界条件	$OUT[ENTRY] = \{ENTRY\}$
交汇运算($\wedge$)	$\cap$
方程式	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{P \in pred(B)} OUT[P]$
初始化设置	$OUT[B] = N$

一个计算支配结点的数据流算法



	OUT0	IN1	OUT1
E	E		
1	N	{E}	{E,1}
2	N	{E,1}	{E,1,2}
3	N	{E,1}	{E,1,3}
4	N	{E,1,3}	{E,1,3,4}
5	N	{E,1,3,4}	{E,1,3,4,5}
6	N	{E,1,3,4}	{E,1,3,4,6}
7	N	{E,1,3,4}	{E,1,3,4,7}
8	N	{E,1,3,4,7}	{E,1,3,4,7,8}
9	N	{E,1,3,4,7,8}	{E,1,3,4,7,8,9}
10	N	{E,1,3,4,7,8}	{E,1,3,4,7,8,10}



```

1) OUT[ENTRY] = v_ENTRY;
2) for (除 ENTRY 之外的每个基本块 B) OUT[B] = T;
3) while (某个 OUT 值发生了改变)
4)   for (除 ENTRY 之外的每个基本块 B) {
5)     IN[B] = \bigwedge_{P \in pred(B)} OUT[P];
6)     OUT[B] = f_B(IN[B]);
  }

```



■ 流图的边的分类

■ 前进边

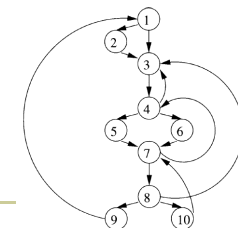
- 从结点m到达m在DFST树中的一个真后代的边
- DFST中的所有边都是前进边

■ 后退边

- 从m到达m在DFST树中的某个祖先的边

■ 交叉边

- 边的src和dest都不是对方的祖先
- 一个结点的子结点按照它们被加入到树中的顺序从左到右排列, 那么所有的交叉边都是从右到左的





流图中的循环

- 回边和可归约性:
- 回边
 - 边 $a \rightarrow b$, 头 b 支配了尾 a
 - 每条回边都是后退边, 但不是所有后退边都是回边
- 如果一个流图的任何优先生成树中的所有后退边都是回边, 那么该流图就是可归约的
 - 可归约流图的DFST的后退边集合就是回边集合
 - 不可归约流图的DFST中可能有一些后退边不是回边, 但是所有的回边仍然都是后退边
- 实践中出现的流图基本都是可归约的



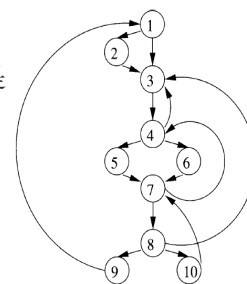
流图中的循环

- 回边和可归约性:
- 回边
 - 边 $a \rightarrow b$, 头 b 支配了尾 a

每条回边都是后退边

但不是所有后退边都是

	OUT0	IN1	OUT1
E	E		
1	N	{E}	{E,1}
2	N	{E,1}	{E,1,2}
3	N	{E,1}	{E,1,3}
4	N	{E,1,3}	{E,1,3,4}
5	N	{E,1,3,4}	{E,1,3,4,5}
6	N	{E,1,3,4}	{E,1,3,4,6}
7	N	{E,1,3,4}	{E,1,3,4,7}
8	N	{E,1,3,4,7}	{E,1,3,4,7,8}
9	N	{E,1,3,4,7,8}	{E,1,3,4,7,8,9}
10	N	{E,1,3,4,7,8}	{E,1,3,4,7,8,10}



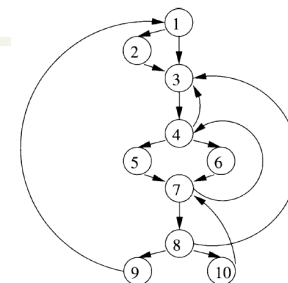
流图中的循环

- 自然循环构造算法:
- 输入: 流图 G 和回边 $n \rightarrow d$;
- 输出: 回边 $n \rightarrow d$ 的自然循环中的所有结点的集合 $loop$;
- 方法
 - $loop = \{n, d\}$, d 标记为visited
 - 从 n 开始, 逆向地对数据流图进行深度优先搜索, 把所有访问到的结点都加入 $loop$, 加入 $loop$ 的结点都标记为visited。搜索过程中, 不越过标记为visited的结点



流图中的循环

- 自然循环示例:
- 回边: $10 \rightarrow 7$
 - $\{7, 8, 10\}$
- 回边: $7 \rightarrow 4$
 - $\{4, 5, 6, 7, 8, 10\}$
 - 包含了前面的循环
- 回边 $4 \rightarrow 3, 8 \rightarrow 3$
 - 同样的头
 - 同样的结点集合 $\{3, 4, 5, 6, 7, 8, 10\}$
- 回边 $9 \rightarrow 1$
 - 整个流图



回边	自然循环
$4 \rightarrow 3$	3,4,5,6,7,8,10
$7 \rightarrow 4$	4,5,6,7,8,10
$8 \rightarrow 3$	3,4,5,6,7,8,10
$9 \rightarrow 1$	1-10
$10 \rightarrow 7$	7,8,10



流图中的循环

- 自然循环的性质:
 - 除非两个循环具有同样的循环头, 他们
 - 要么互不相交 (分离的)
 - 要么一个嵌套于另一个中
 - 最内层循环
 - 不包含其它循环的循环
 - 通常是最需要进行优化的地方



- (7分) 请将图中给定的三地址码格式的程序划分为程序基本块。建议采用课程中使用的方框中填写代码的形式划分这个流程图, 当然, 也可以采用基本块的格式为 $B_n = \{(l_1), \dots\}$ 。例如, 如果第4行和第10行三地址码在第1个基本块中, 那么该基本块可以表示成 $B_1 = \{(4), (10)\}$ 。然后, 以你给出的基本块为结点, 画出程序控制流程图。↵
- 注: 控制流图中还需要补充 Entry 和 Exit 结点, 在本题中这两个结点都是唯一的。↵

```

(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z * z goto (13)
(17) g = f + 1
(18) h = g * 2
    
```



2. (8分) 基于你在第1小题中构建的控制流图, 分别尝试使用下面的优化技术对程序进行优化, 描述优化前后的三地址码变动, 可以通过画图表示, 也可以通过上述的

$B_n = \{l_1, \dots\}$ 形式表示。
a. 公共子表达式删除
b. 死代码删除
c. 循环优化

```
(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z*z goto (13)
(17) g = f + 1
(18) h = g * 2
```



a.公共子表达式删除: 在代码中, 第16行 条件检查中 $z * z$ 被重复计算。我们可以在第3行后添加一条新指令, 计算 $t' = z * z$, 然后在第16行使用 t' 来替代 $z * z$ 。减少每次循环对 $y * y$ 的重复计算。
b.死代码删除: 第8行 (说第18行、第17行也对)
c.循环优化: 第15行的 $f=x-y$ 在每次进入循环时都会重新计算, 如果 x 和 y 在循环中不变, 那么这个计算可以移到循环外进行。这样, 可以减少每次循环中的计算量 (说第16行的 $f=-3 < z*z$ 永假也对。)

```
(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z*z goto (13)
(17) g = f + 1
(18) h = g * 2
```

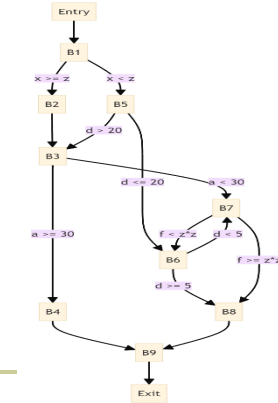


表格如下:

指令	R1	R2	t1	t2	t3	i	x	a[i+1]
LOAD R1, <u>i</u>	<u>i</u>					<u>i</u> , R1	x	a[i+1]
ADD R1, R1, #1	t1		R1			<u>i</u>	x	a[i+1]
MULT R2, R1, #4	t1	t2	R1	R2		<u>i</u>	x	a[i+1]
LOAD R1, a(R2)	a[i+1], t3, x	t2		R2	R1	<u>i</u>	R1	a[i+1], R1
STORE x, R1	a[i+1], t3, x	t2		R2	R1	<u>i</u>	x, R1	a[i+1], R1



B1: { (1), (2), (3), (4) }
B2: { (5), (6) }
B3: { (7) }
B4: { (8), (9) }
B5: { (10), (11), (12) }
B6: { (13), (14) }
B7: { (15), (16) }
B8: { (17) }
B9: { (18) }



```
(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z*z goto (13)
(17) g = f + 1
(18) h = g * 2
```



假设有两个可用的寄存器 R1 和 R2, 使用简单代码生成算法,

得分	
评分人	

将下面的三地址码转换为机器代码。

```
t1 = i + 1
t2 = t1 * 4
t3 = a[ t2 ]
x = t3
```

给出每条三地址码生成机器代码后的寄存器描述符和地址描述符 (请在表格左边空白

部分写出生成的机器代码, 并在表格中补充写入对应的描述符内容。设数组的每个元素

占 4 个字节, 表格长度可自行添加)。

附机器代码格式:

取数指令: LOAD R, var // 将内存空间中变量 var 的值存入寄存器 R 中。

存数指令: STORE var, R // 将寄存器 R 中的值存入变量 var 的内存空间中。

双目运算指令: OP dst, src1, src2 // 使用 OP 对寄存器 src1 和 src2 中的值进行计算, 结果存入寄存器 dst 中 (OP 包括 ADD、SUB、MULT、DIV)。

表格如下:

指令	R1	R2	t1	t2	t3	i	x	a[i+1]
----	----	----	----	----	----	---	---	--------

考虑右图所示流图, 完成以下题目。

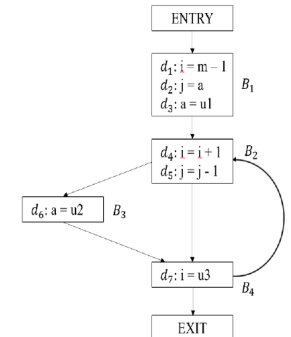
得分	
评分人	

1. 如果要对其进行到达值分析, 请写出各个基本块的 def 和 use 集合;

2. 请对流图进行活跃变量分析, 写出分析算法每次迭代的结果。

(提示: 注意活跃变量分析的方向)

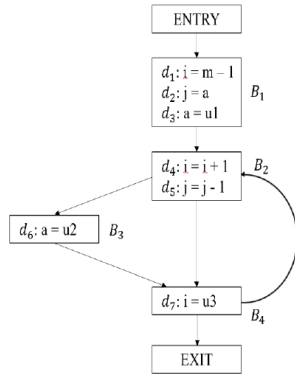
1. $USE_{B1} = \{ \}$
 $DEF_{B1} = \{ \}$
 $USE_{B2} = \{ \}$
 $DEF_{B2} = \{ \}$
 $USE_{B3} = \{ \}$
 $DEF_{B3} = \{ \}$
 $USE_{B4} = \{ \}$
 $DEF_{B4} = \{ \}$





2.

	$OUT[B]^i$	$IN[B]^i$	$OUT[B]^j$	$IN[B]^j$	$OUT[B]^i$	$IN[B]^i$
B_4						
B_1						
B_2						
B_3						



九、简答题（共 1 题，共 10 分）

答案：

1. (5 分)

$USE_{B1} = \{m, a, u1\}$

$DEF_{B1} = \{i, j\}$

$USE_{B2} = \{i, j\}$

$DEF_{B2} = \{j\}$

$USE_{B3} = \{u2\}$

$DEF_{B3} = \{a\}$

$USE_{B4} = \{u3\}$

$DEF_{B4} = \{i\}$

2. (5 分)

	$OUT[B]^i$	$IN[B]^i$	$OUT[B]^j$	$IN[B]^j$	$OUT[B]^i$	$IN[B]^i$
B_4	-	u3	i,j,u2,u3	j,u2,u3	i,j,u2,u3	j,u2,u3
B_3	u3	u2,u3	j,u2,u3	j,u2,u3	j,u2,u3	j,u2,u3
B_2	u2,u3	i,j,u2,u3	j,u2,u3	i,j,u2,u3	j,u2,u3	i,j,u2,u3
B_1	i,j,u2,u3	m,a,u1,u2,u3	i,j,u2,u3	m,a,u1,u2,u3	i,j,u2,u3	m,a,u1,u2,u3



十、简答题（共 1 题，共 10 分）

考虑下图所示代码片段，完成下列题目：

- 请画出代码片段中的各个基本块及其流图，基本块用矩形表示，矩形中需要包括对应的代码片段，基本块用 B_i 命名，即 B_1 表示第一个基本块；
- 求出流图中自然循环及其结点集合；
- 请对流图给出至少三条优化建议（指出哪些语句可以进行哪些优化即可，不需要画出优化后的流图）。

得分	
评分人	

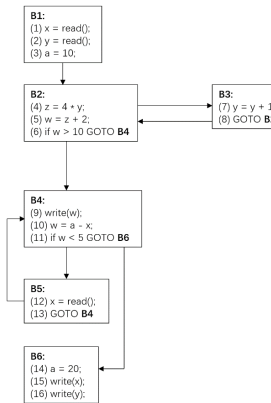
```
(1) x = read();
(2) y = 4;
(3) a = 10;
(4) z = x * y;
(5) w = z + 2;
(6) if w > 10 GOTO (9)
(7) y = y + 1;
(8) GOTO (4)
(9) write(w);
(10) w = a - x;
(11) if w < 5 GOTO (14)
(12) x = read();
(13) GOTO (9)
(14) a = 20;
(15) write(x);
(16) write(y);
```



十、简答题（共 1 题，共 10 分）

答案：

1. (3 分)



2. (4 分)

回边 B_3 - B_2 : $\{B_2, B_3\}$

回边 B_5 - B_4 : $\{B_4, B_5, B_6\}$

3. (3 分)

常量折叠：语句 10 中 a 用 10 代替

死代码消除：语句 14

强度消减：语句 4

```
(1) x = read();
(2) y = 4;
(3) a = 10;
(4) z = x * y;
(5) w = z + 2;
(6) if w > 10 GOTO (9)
(7) y = y + 1;
(8) GOTO (4)
(9) write(w);
(10) w = a - x;
(11) if w < 5 GOTO (14)
(12) x = read();
(13) GOTO (9)
(14) a = 20;
(15) write(x);
(16) write(y);
```